Module 1 — Git and GitHub

Vistula University · IT Fundamentals · Theory handout

Vistula University · IT Fundamentals

Learning Objectives

After this module you will be able to:

- Explain what version control is and why developers use it.
- Explain the difference between Git and GitHub.
- Create a repository, clone it to your computer, and push changes back.
- Use the basic Git commands: `git add`, `git commit`, `git push`, `git pull`.
- Create a branch, work on it, and merge it back.
- Open a pull request and review a partner's code.
- Understand what a merge conflict is and how to fix a simple one.
- Follow a simple team workflow based on feature branches and pull requests.

What Is Version Control?

Imagine you write an essay and save it many times: `essay.doc`, `essay2.doc`, `essay_final.doc`, `essay_final_REALLY_final.doc`. After a few days you do not know which file is the newest, and you cannot easily go back to yesterday's version.

Version control is a system that remembers every version of your files for you. You do not save copies with new names. Instead, you tell the system: "remember this state now". Later, you can always see what changed, who changed it, and go back to an older version if something breaks.

Why Do Developers Use Git?

Git is the most popular version control system in the world. It was created in 2005 by Linus Torvalds, the creator of Linux. Developers use Git because it:

- **Keeps history.** Every change is saved with a comment, so you can see the story of the project.
- **Lets you undo mistakes.** If new code breaks the project, you can go back to the last working version.
- **Makes teamwork safe.** Many people can work on the same project at the same time without destroying each other's work.
- **Works offline.** Git works on your own computer. You only need the internet when you share your work.

Git versus GitHub

Students often confuse these two things. They are different:

- **Git** is a *program* that runs on your computer. It is free. You use it in the terminal (a window where you type commands).

- **GitHub** is a *website* (github.com) that stores your Git projects in the cloud. It lets people share code, discuss it, and work together.

A simple comparison: Git is like Microsoft Word on your computer; GitHub is like OneDrive, where your documents are stored online and can be shared with others. You can use Git without GitHub, but they work best together.

Repository, Working Directory and Staging Area

A **repository** ("repo" for short) is your project folder plus the full history of all its versions. It lives on your computer and can also live on GitHub.

Git uses three "places" for your files. This is the most important idea in this module:

1. **Working directory** — the normal folder on your computer. You edit files here. Git does not remember these changes yet.
2. **Staging area** — a waiting room. Here you put exactly the changes you want to save in your next version.
3. **Repository (history)** — when you "commit", the staged changes are saved into the history.

Why a staging area? Because you may change five files, but want to save only two of them now. First you put the two files into the staging area, then you commit.

The Basic Commands: Clone, Add, Commit, Push, Pull, Fetch

Here are the commands you will use most often. Type them in the terminal, inside your project folder.

```
git clone https://github.com/your-name/my-project.git
```

Copies a repository from GitHub to your computer. You do this once, at the start.

```
git add index.html
```

Puts the file `index.html` into the staging area. You can also use `git add .` to stage all changed files.

```
git commit -m "Add a short description of the project"
```

Saves the staged changes into the history. The `-m` part is your short message saying what you did.

```
git push
```

Sends your commits from your computer to GitHub, so other people can see them.

```
git pull
```

Takes new commits from GitHub and puts them into your local repository. Do this before you start working, so you have the newest version.

```
git fetch
```

Downloads new information from GitHub but does not change your files yet. It is like “look, but do not touch”. `git pull` = `git fetch` + adding the changes to your files.

The normal cycle of daily work looks like this: **pull** → **edit files** → **add** → **commit** → **push**. Repeat.

Branches and Merging

Imagine two people editing the same document at the same time. It will be chaos. Git solves this with **branches**.

A **branch** is an independent line of work. The main branch is usually called `main` (or `master` in older projects). When you create a new branch, you copy the current state of the project and can change it freely — the `main` branch stays safe.

A typical branch name describes the task: `feature-contact-form` or `fix-typo-on-homepage`.

```
git branch feature-colors
```

Creates a new branch called `feature-colors`. It does not switch to it yet.

```
git checkout feature-colors
```

Switches to the branch `feature-colors`. Now your edits are on this branch only. (Tip: `git checkout -b feature-colors` creates and switches in one step.)

```
git merge feature-colors
```

While on `main`, this command copies the changes from `feature-colors` into `main`.

Merging means joining the changes from one branch into another. After a merge, both branches contain the same work.

Pull Requests and Code Review

A **pull request** (PR) is a request on GitHub: “please take my changes from this branch and put them into `main`”. It is not a Git command — it is a page on the GitHub website. On this page, your teammates can see exactly what you changed, line by line, and write comments.

Code review means that another person reads your code before it is merged. They check: Is it correct? Is it clear? Does it follow the team’s rules? The reviewer can approve the pull request or ask for changes. Code review catches mistakes early and helps the whole team learn.

A simple rule for this course: **never push directly to `main` — always work on a branch and open a pull request.**

Merge Conflicts

Sometimes Git cannot join two branches automatically. This happens when **two people change the same line of the same file** in different ways. Git does not know which version is correct, so it asks you to decide. This situation is called a **merge conflict**.

In the file, Git marks the conflict like this:

```
<<<<<<< HEAD
Welcome to our project.
=======
Welcome to our wonderful project!
>>>>>>> feature-title
```

The text between `<<<<<<< HEAD` and `=======` is your version. The text between `=======` and `>>>>>>> feature-title` is the other version. To resolve the conflict:

1. Open the file and choose the final text (your version, their version, or a mix). Keep only one version.
2. Delete the `<<<<<<<`, `=======` and `>>>>>>>` lines.
3. Stage the file: `git add index.html`.
4. Finish with a commit: `git commit -m "Resolve merge conflict"`.

Conflicts sound scary, but small ones are easy. The best protection is to pull often and to keep changes small.

Why a Team Needs a Clear Git Workflow

If every team member invents their own way of working, the repository becomes a mess. A **Git workflow** is an agreed set of rules: which branch is the main one, how new work starts, and how changes are merged. A clear workflow prevents lost work and constant conflicts.

A simple and very popular workflow is **feature branches + pull requests**:

1. The team agrees that `main` must always work. Nobody pushes to `main` directly.
2. For each new task, a developer creates a branch, for example `feature-login-page`.
3. The developer works on this branch and pushes it to GitHub.
4. The developer opens a pull request.
5. Another team member reviews the code and approves it.
6. The pull request is merged into `main`, and the branch is deleted.

With this workflow, `main` is always clean, every change is reviewed, and two people almost never edit the same files at the same moment.

Key Terms (Glossary)

Term	Simple meaning (English)	Po polsku
version control	a system that remembers every version of your files	system kontroli wersji
repository	a project folder with its full history	repozytorium
working directory	the folder where you edit your files	katalog roboczy
staging area	the waiting room for changes before a commit	obszar przejściowy
commit	a saved version of the project with a message	zapis (commit)
clone	to copy a repository from GitHub to your computer	sklonować
push	to send your commits to GitHub	wypchnąć
pull	to download new commits from GitHub	pobrać
fetch	to download changes from GitHub without applying them	pobrać (bez stosowania zmian)

Term	Simple meaning (English)	Po polsku
branch	an independent line of work in the project	gałąź
merge	to join changes from one branch into another	scalić
pull request	a request to merge your branch, with discussion and review	żądanie scalenia (pull request)
code review	another person reads your code before merging	przegląd kodu
merge conflict	a situation when Git cannot join changes automatically	konflikt scalania

Summary

- Version control remembers every version of your project, so you can go back in time.
- Git is the program on your computer; GitHub is the website that stores repositories online.
- Files move through three places: working directory → staging area → commit in the history.
- The daily cycle is: pull, edit, `git add`, `git commit`, `git push`.
- A branch is a safe, independent line of work; `main` should always work.
- A pull request lets teammates review your code before it is merged.
- A merge conflict happens when two people change the same line; you fix it by choosing one version.
- Teams agree on a workflow — for example, feature branches plus pull requests.

> **Need help?** The official GitHub Docs at https://docs.github.com are a great source of help. They have guides for beginners, and you can search them any time during class or homework.